claude-windows / 8d4866bf-8edd-4da9-909e-022ddee12675

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\8d4866bf-8edd-4da9-909e-022ddee12675\subagents\workflows\wf_684a0618-ccb\agent-a9c5a45cd19cad3ec.jsonl`
- SHA-256: `7b28a796074ebd18949551fdfd3998ac3df05e472ab751412d3b518357e9a701`
- Source modified: `2026-06-19T19:19:04+00:00`
- Imported at: `2026-07-08T16:00:03+00:00`
- project: `wf_684a0618-ccb`
- session_id: `8d4866bf-8edd-4da9-909e-022ddee12675`

Transcript

1. user (2026-06-19T19:17:29.036Z)

THE IDEA TO ASSESS: a desktop app OR web UI where the user points KhelSutra at an ONLINE storage location they already own/pay for ? a Google Cloud Storage bucket, Google Drive, OneDrive/Microsoft 365, Dropbox, S3, etc. KhelSutra then WATCHES that folder, pulls/streams new recordings as they arrive, and AUTO-prepares highlights (same processing as the local/desktop path). "Bring your own storage + autosync": footage lands in their cloud ? reels appear automatically, zero manual upload.

You are assessing how this idea fits ONE area of the KhelSutra (badminton-highlight-indexer) docs. Read: docs/DESKTOP_APP_PLAN.md and docs/UI_ALPHA_EXECUTION_PLAN.md. Does the desktop-app plan consider a cloud-storage SOURCE (not just SD card / local files) and a watch/autosync daemon? Does the web UI alpha consider connecting external storage? Where would a connector + background sync live in each surface?

Be evidence-based and honest ? quote real anchors, don't invent. If the doc does NOT anticipate the idea, say so plainly and note what it WOULD take. Return the structured object.

2. assistant (2026-06-19T19:17:31.469Z)

I'll read both docs to assess how the BYO-storage + autosync idea fits.

3. user (2026-06-19T19:17:32.534Z)

1	# Cross-Platform Desktop App ? "Plug in your card, get your highlights"
2
3	> **Status:** `DRAFT` ? `owner-gated` (scoping doc only; **nothing is built**) · **Owner:** avidullu · **Created:** 2026-06-18 · **Last updated:** 2026-06-18
4	> **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when DONE)
5	> **Tracking anchors:** §7 progress tracker is the source of truth; indexed in `docs/README.md`; pointer in memory `current-state` (and `docs/NEXT_STEPS.md` once the owner greenlights P0).
6	> **Relation to existing docs:** **peer-of** [`PLATFORM_ARCHITECTURE.md`](PLATFORM_ARCHITECTURE.md) (this is the **`LocalDesktop` ComputeBackend** made into a product) · **realizes the L0 tier** of [`VIDEO_LOCALITY_MODEL.md`](VIDEO_LOCALITY_MODEL.md) · **consumes** [`OWNED_MODEL_IMPLEMENTATION_PLAN.md`](OWNED_MODEL_IMPLEMENTATION_PLAN.md) (the smaller/export-clean model is one answer to the compute crux) · bound by [`COMMERCIALIZATION.md`](COMMERCIALIZATION.md) licensing guardrails.
7	> **Honesty note:** every claim is tagged `[verified]` (checked against code/measured), `[researched]` (needs sign-off), or `[design]` (proposed). **The entire build is `[design]`** ? this doc scopes; it does not ship. Not legal advice.
8
9	---
10

0. TL;DR

Package the mature local pipeline (WASB shuttle detector + improved windowing + ffmpeg stitch, already runnable fully-offline via [`tools/generate_highlights.py --skip-ai-handoff`](../tools/generate_highlights.py)) as a **cross-platform desktop app** for **non-technical recreational players**: plug in the camera's USB/SD card, click once, get back **only the highlight reels** ? fully on-device, **no upload**, **no copy of the GBs of 4K originals**. This is the product surface that fits the "local, cheap, efficient" thesis and the L0 ("nothing leaves the machine") tier of `VIDEO_LOCALITY_MODEL.md`.

The make-or-break unknown is compute on a typical enthusiast laptop ? which often has **no discrete GPU**. WASB runs at **~3.4× real-time on a laptop RTX 2070 Super** `[verified, measured]`; on CPU it is far slower, and Apple Silicon has **no CUDA** at all. **P0 is a feasibility spike that measures CPU and Apple-MPS throughput for a ~10-min clip before any app is built.** If CPU-only is unacceptable on commodity hardware, the path needs a smaller/faster owned model (ties `OWNED_MODEL_IMPLEMENTATION_PLAN`) or a tiered "fast-if-GPU, slow-but-works-if-CPU" design. **Everything past P0 is gated on that result.**

1. Problem & goal

Why now / the user. Target users are **recreational sports enthusiasts, not tech-savvy**. Their match videos are **huge** (GBs of 4K). The cloud-upload product surface fails them on every axis: slow on a typical Indian uplink (`VIDEO_LOCALITY_MODEL.md` §0: 10 GB ? ~1 h at 20 Mbps, ~4.5 h at 5 Mbps), bandwidth-costly, privacy-fraught, and it bloats both our storage and their drives. The thing the customer actually wants back is **kilobytes of timestamps ? a highlight reel** (`VIDEO_LOCALITY_MODEL.md` §1: *"timestamps are the product; the video is dead weight"*).

The concrete outcome ("good" looks like this):
1. User plugs the GoPro/phone SD card or camera USB into their laptop.
2. The app detects the card, lists the match videos on it.
3. User clicks **Generate Highlights**.
4. The app makes an on-device proxy, runs WASB detection + windowing + stitches the reel ? **100% locally**.
5. The reel(s) land in a user-chosen folder. **The originals are never copied off the card and never bloat any drive.**

No account, no upload, no cloud dependency in the default path. Gemini stays an **optional** "better brain if you want it" toggle ? never required, never bundled as a hard dependency, never a training source (`COMMERCIALIZATION` C4).

Read to get current: `VIDEO_LOCALITY_MODEL.md` (the L0/locality framing + OQ5 packaging seed), `PLATFORM_ARCHITECTURE.md` §3.2 (the ComputeBackend seam), [`backend/pipeline/detectors/base.py`](../backend/pipeline/detectors/base.py) (the detector ABC that already anticipates a Linux-native runner), `tools/generate_highlights.py` (the local entrypoint this app wraps).

2. Decisions locked

#	Decision	Source / date	Implication
D1	**Default path is 100% on-device, no upload, no Gemini.** The reel is produced from the local pipeline (`--skip-ai-handoff`).	This doc · 2026-06-18; matches `VIDEO_LOCALITY_MODEL` L0	The app must never *require* a network call. Gemini is an opt-in toggle only.
D2	**Never copy the originals.** The app reads source video in place (off the card / a chosen folder) and writes **only** reels + a tiny report to a user folder.	This doc; mirrors `generate_highlights.py` non-destructive `_atomic_publish` discipline	No "import library" that duplicates 10 GB files. Detection runs on a proxy; stitch reads the original in place.
D3	**Drop the WSL2 dependency; run the detector natively per-OS.**	This doc; enabled by the `DetectorRunner` ABC `[verified]`	Windows-native (no WSL), Linux+NVIDIA (CUDA directly), macOS (MPS or CPU). The WSL adapter stays for the current dev box but is **not** the shipped path.
D4	**Commercial-clean licensing for everything bundled: MIT / Apache-2.0 / BSD only** (code, weights). ffmpeg ships as an **LGPL build** (no GPL-only encoders like x264).	`COMMERCIALIZATION` (ratified 2026-06-09); `VIDEO_LOCALITY_MODEL` OQ5 (*"ffmpeg LGPL build =

licensing-clean"*) | WASB-SBDT weights are MIT `[verified]` (see `tracknet-wasb-wsl2-decision`). The stitch must use openh264 (BSD) or OS hardware encoders, **not** statically-linked x264. See §3.4. |
39 | D5 | **Reuse the existing pipeline verbatim; add only the shell + a native runner + device-IO.** | `backend/pipeline/detectors/base.py` reuse contract `[verified]` | No re-architecture of windowing/stitch. New code = native runner, USB/SD detection, app shell, packaging. |
40 | D6 | **Feasibility is gated on P0.** No app shell, no packaging work starts until the compute spike returns a verdict. | This doc; `decision-rigor-norm` (NUMBERS INFORM, FOUNDATIONS DECIDE) | P0 is the only un-gated deliverable; P1?P3 are owner-greenlit *after* P0 numbers. |
41
42 ## 3. Foundation ? research / prior art `[design]` / `[researched]`
43
44 ### 3.1 The compute reality (the crux) ? what we know vs must measure
45 - **WASB on a discrete laptop GPU is acceptable** `[verified, measured]`: on the owner's **RTX 2070 Super Max-Q (8 GB)**, WASB inference is the pipeline driver at **~3.2?3.4× real-time** (?3.4 GPU-min per footage-min, ~0.055 s/frame), peak VRAM ~5.7?6.0 GB; proxy (NVENC) ~0.2× RT; 4K stitch (CPU libx264) ~16 s/rally; end-to-end ?5× clip length. Source: `current-state.md` `COST_SCALING.md` checkpoint (6 Boxhill clips, 2026-06-18).
46 - **The typical enthusiast laptop has NO discrete GPU.** A 10-min 4K clip at 3.4× RT is ~34 GPU-min *with* a 2070S. On integrated graphics / CPU-only the wall-clock is unknown and could be 10×+ worse ? possibly **hours per clip**, which would kill the one-click promise.
47 - **Apple Silicon has no CUDA.** WASB/TrackNet weights are PyTorch; PyTorch MPS (Metal) backend *may* run them, but op-coverage and speed on the WASB model are **unverified** `[design]`. CPU fallback on Apple is also unverified.
48 - **Owned-model tie-in.** `OWNED_MODEL_IMPLEMENTATION_PLAN.md` already states the on-device answer: *train in Python ? **export across the boundary** (ONNX / Core ML / ExecuTorch / GGUF) ? a thin device shell runs the artifact with zero Python*, with the standing constraint to **train with export-clean ops**. A smaller, exported, quantized shuttle detector is a credible answer if the CPU/MPS numbers on the current WASB weights are bad. **This makes P0 partly a referendum on whether the desktop product needs the owned-model track.**
49
50 ### 3.2 The de-WSL native runner is already designed-for `[verified]`
51 The detector layer was deliberately built to make this port low-regret. [`backend/pipeline/detectors/base.py`](../backend/pipeline/detectors/base.py) documents, verbatim, an **"Optional Linux-native path (the main de-risk goal)"**: a `LinuxNativeRunner`/`ONNXRunner` that *"Skip[s] all wsl / _stage / conda activate, load[s] weights locally (torch or onnx)? write[s] the exact same CSV format so `trajectory_to_action_windows` works verbatim."* The WSL plumbing lives in a separate `WslCommandMixin`, explicitly so a native runner *"must not inherit WSL plumbing."* The actual inference math is already isolated in [`wasb_infer.py`](../backend/pipeline/detectors/wasb_infer.py) (torch + the WASB-SBDT modules, emitting `Frame,Visibility,X,Y`). **So the port is: lift `wasb_infer.py` out of WSL, load weights with native torch/onnxruntime per-OS, keep the identical CSV contract.** The windowing brain (`trajectory_to_action_windows`) and stitch are untouched.
52
53 ### 3.3 Compute backends to evaluate in P0
54 | Path | OS | Toolkit | Status |
55 |---|---|---|---|
56 | CUDA-native | Windows-native, Linux+NVIDIA | torch+CUDA (no WSL) | proven in WSL `[verified]`; native = port |
57 | Apple MPS | macOS (Apple Silicon) | torch MPS / Core ML export | **unverified ? P0 must measure** |
58 | CPU | all (the no-GPU laptop) | torch-CPU / **onnxruntime** (often fastest on CPU) | **unverified ? P0 must measure; the gating number** |
59 | Smaller/exported model | all | ONNX/Core ML/ExecuTorch, quantized | deferred to owned-model track; P0 informs whether it's *needed* |
60
61 ### 3.4 App-shell prior art & licensing landscape `[researched]`
62 - **App shell options:** **Tauri** (Rust + system webview; ~3?10 MB binaries; MIT/Apache) vs **Electron** (bundled Chromium; ~100?150 MB; MIT) vs **CLI + installer** (wrap `generate_highlights.py`; smallest effort). All shell licenses are clean.
63 - **Backend packaging:** the Python/FastAPI backend bundles via **PyInstaller** or a frozen venv as a local "sidecar." Packaging was already modernized ? **PEP 621 `pyproject.toml` + an `indexer` console entry point** shipped (`DEFERRED_HARDENING_PLAN` #7, DONE, PR #167)

`[verified]` ? so a CLI MVP has a real head start.
64   - **ffmpeg licensing (a genuine landmine):** ffmpeg is LGPL/GPL. The current stitch path uses **libx264 (GPL)**. Redistributing that inside an installer would impose GPL on the bundle. **Mitigation (D4):** ship an **LGPL ffmpeg build** using **openh264 (BSD)** or the OS hardware encoder (NVENC / Apple **VideoToolbox** / Linux **VAAPI**) ? the desktop app should prefer hardware H.264 anyway (faster, no CPU stitch tax).
65   - **Weights:** WASB-SBDT = MIT `[verified]` (`tracknet-wasb-wsl2-decision`), redistributable.
66
67   ## 4. Design / architecture `[design]`
68
69   ### 4.1 The one-click flow (and what code each step reuses)
70   ```
71   [card inserted]
72   ? (NEW) per-OS device-insert watcher ? mount path
73   ?
74   [list videos on the card]   ? glob video exts; reuse _VIDEO_EXTS from generate_highlights.py
75   ?
76   [make on-device proxy]   ? ffmpeg downscale (hardware encoder); reuse the proxy + --detect-from split (#205)
77   ?
78   [detect: WASB native]   ? (NEW) NativeWasbRunner (DetectorRunner subclass) ? NO WSL; same CSV contract
79   ?
80   [windowing ? rallies]   ? REUSE trajectory_to_action_windows + improved milestone windowing (R1: cap=6 + R4)
81   ?
82   [stitch reel from ORIGINAL] ? REUSE VideoCompiler (backend/pipeline/stitcher/compiler.py); read original in place
83   ?
84   [export reel + tiny report to a user folder]   ? reuse _atomic_publish non-destructive write; NEVER copy originals
85   ```
86   The detector runs on a **proxy**; the stitch reads the **original on the card** (the `--detect-from` split, already shipped `[verified]` in `generate_highlights.py`). Rally timestamps transfer 1:1 because the proxy shares the timeline (the collector frame-identical proxy contract, `VIDEO_LOCALITY_MODEL` §1).
87
88   ### 4.2 Reuse map ? what's NEW vs REUSED
89 | Concern | Status | Code |
90 |---|---|---|
91 | Trajectory?rally windowing | **REUSE** `[verified]` | `tracknet_runner.trajectory_to_action_windows` (model-agnostic staticmethod) |
92 | Stitch / reel compile | **REUSE** `[verified]` | `backend/pipeline/stitcher/compiler.py::VideoCompiler` (pure ffmpeg) |
93 | Local orchestration (proxy?detect?stitch, non-destructive, resumable, `--skip-ai-handoff`, `--detect-from`) | **REUSE** `[verified]` | `tools/generate_highlights.py` |
94 | Detector runner contract | **REUSE** `[verified]` | `backend/pipeline/detectors/base.py::DetectorRunner` ABC |
95 | WASB inference math | **REUSE (lift out of WSL)** `[verified]` | `backend/pipeline/detectors/wasb_infer.py` |
96 | **Native (de-WSL) WASB runner, per-OS** | **NEW** `[design]` | `NativeWasbRunner` ? sibling of `wasb_runner.py`, no `WslCommandMixin` |
97 | **USB/SD insert detection + video listing** | **NEW** `[design]` | per-OS: Win WMI/`WM_DEVICECHANGE`, macOS DiskArbitration, Linux udev/udisks2 |
98 | **App shell / one-click UI** | **NEW** `[design]` | Tauri vs Electron vs CLI (§3.4; P2 decides) |
99 | **Packaging + code-signing per OS** | **NEW** `[design]` | PyInstaller/MSIX (Win), .dmg+notarize (mac), AppImage/Flatpak (Linux) ? P3 |
100
101   ### 4.3 Where this sits in the platform architecture
102   This **is** the `local` / `LocalDesktop` **ComputeBackend** of `PLATFORM_ARCHITECTURE.md` §3.2, realized as a shippable product instead of a server worker. The control-plane/data-plane split degenerates cleanly to "all on one machine." Nothing here

forecloses the hosted SaaS path ? it's the privacy-max end of the same spectrum, and a genuine differentiator (no competitor offers true on-device compute except SwingVision, which proves it's commercially viable ? `PLATFORM_ARCHITECTURE` §intro).
103
104   ### 4.4 App-shell recommendation (to confirm at P2)
105   Lean: **start with a CLI + installer MVP** (wraps the existing `indexer`/`generate_highlights.py` entry point ? fastest way to validate packaging, signing, USB-detect, and the native runner end-to-end on all three OSes), then **graduate to a Tauri GUI** for the non-technical user (tiny download fits the "huge-video laptops, slim app" frame; the Python backend runs as a sidecar the Tauri front-end calls over localhost). Electron is the fallback if webview inconsistencies bite. This mirrors the repo's "simplest thing first, graduate on evidence" bias.
106
107   ## 5. Risk table `[design]`
108 | Risk | Severity | Mitigation / where decided |
109 |---|---|---|
110 | **CPU/MPS too slow on a no-GPU laptop** (kills the one-click promise) | **make-or-break** | **P0 spike measures it before anything is built** (§6, §7-P0). Fallbacks: tiered fast/slow UX, or the owned smaller model. |
111 | ffmpeg GPL contamination via libx264 in the installer | high (legal) | D4: LGPL build + openh264/hardware encoders only. |
112 | Code-signing friction (mac notarization mandatory; Win SmartScreen without EV cert) | medium | P3 scopes certs/notarization per OS; budget the Apple Developer Program + a Windows cert. |
113 | Privacy regressions (the on-device promise is the *selling point*) | medium | D1/D2: no upload, no original-copy, no required network. Make the "nothing leaves your machine" guarantee testable (assert no outbound calls in default mode). |
114 | Per-OS device-insert flakiness / removable-media edge cases (card pulled mid-run) | medium | P1/P2 handle gracefully; never write to the card; fail safe. |
115 | Support burden for non-technical users (drivers, "it didn't detect my card") | medium | favors a polished GUI + clear errors (reuse the actionable-error discipline already in the WSL adapter). |
116
117   ## 6. Honest limits ? what this does NOT do
118   - **A DRAFT/DONE status here proves nothing about runtime feasibility.** The doc is a scope; only the P0 numbers decide whether the product is buildable on commodity hardware.
119   - **If CPU-only is too slow on a typical laptop, the simple "wrap the pipeline" path does not exist** ? it becomes "ship a smaller owned model" (a much larger effort, ties `OWNED_MODEL_IMPLEMENTATION_PLAN`) or "GPU-tier only" (shrinks the addressable market). The doc names this honestly rather than assuming the wrap is free.
120   - **No accuracy claim changes.** The desktop app inherits exactly the current rally-detection quality (R1 milestone: precision is still the open gap per `RALLY_DETECTION_GAP_CLOSURE.md`); packaging does not improve detection.
121   - **Gemini-quality reels need the optional online toggle.** The fully-local default uses `--skip-ai-handoff`; users wanting the Gemini brain accept that ?1280px chunks leave the machine (consent language, `VIDEO_LOCALITY_MODEL` OQ3).
122   - **macOS support is contingent on P0's MPS/CPU verdict** ? it is not assumed.
123
124   ## 7. Deliverables & progress tracker   ? **source of truth**
125   Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. **One small PR per row**, branched from `origin/master`, no stacking. Update the row (status + PR link) as work lands. **Everything below is owner-gated; only this doc (the scoping PR) is in flight today.**
126
127 | ID | Deliverable | Depends on | Gated? | Status | PR |
128 |----|-------------|-----------|--------|--------|----|
129 | **D0** | **This scoping doc** (`DESKTOP_APP_PLAN.md` + README index) | ? | No | ? | *(this PR)* |
130 | **P0** | **Compute-feasibility spike (make-or-break).** Measure WASB throughput for a ~10-min clip on: (a) CPU-only (torch-CPU & onnxruntime), (b) Apple MPS, (c) confirm CUDA-native (no WSL). Output: a wall-clock table + a go/no-go verdict + "do we need a smaller model?" recommendation. **No app code.** | D0 | **Owner greenlight** | ? | ? |
131 | **P1** | **De-WSL native detector port** (`NativeWasbRunner`, per-OS): CUDA-native (Win/Linux), MPS/CPU fallback, identical `Frame,Visibility,X,Y` CSV; reuse windowing verbatim; healthcheck per backend. | P0 verdict | Yes | ? | ? |
132 | **P2** | **Desktop app shell + one-click flow**: USB/SD detect ? list ? proxy ? native

detect ? stitch ? export; shell choice (CLI MVP ? Tauri) confirmed; "no upload / no original-copy" enforced & testable. | P1 | Yes | ? | ? |
133 | **P3** | **Packaging + code-signing installers** per OS (Win MSIX/Authenticode, mac .dmg + Developer-ID notarization, Linux AppImage/Flatpak); LGPL ffmpeg bundling; MIT/Apache/BSD audit of the bundle. | P2 | Yes | ? | ? |
134
135   ## 8. Open questions ? owner / external
136   **Blocking gates (owner decides before the dependent phase):**
137   1. **(gates P1?P3) Greenlight P0?** ? the spike is cheap and decides everything. Recommend: yes, run P0 next.
138   2. **(gates P1) If CPU is too slow, which fork?** tiered GPU-fast/CPU-slow UX · invest in a smaller exported owned model (`OWNED_MODEL_IMPLEMENTATION_PLAN`) · GPU-tier-only product (narrower market).
139   3. **(gates P2) App shell:** confirm CLI-MVP-then-Tauri, or go straight to a GUI / Electron.
140   4. **(gates P3) Distribution & signing budget:** Apple Developer Program (mandatory for notarization), a Windows code-signing cert (EV for clean SmartScreen) ? owner cost decision.
141
142   **Nice-to-knows:**
143   5. OS coverage priority for v1 (Windows-first? given the owner's box and the academy beachhead).
144   6. Auto-update strategy (Tauri updater / Sparkle / none for the pilot).
145   7. Does the desktop app reuse the optional Gemini toggle at all, or stay strictly local for v1?
146
147   ## 9. Definition of done
148   This subproject is DONE (? flip Status, archive) when:
149   - [ ] **P0** produced a measured throughput table (CPU / MPS / CUDA-native) and an owner-accepted go/no-go verdict (incl. whether a smaller model is required). *(P0 alone is a valid stopping point if the verdict is "not feasible without the owned model" ? then this doc hands off to `OWNED_MODEL_IMPLEMENTATION_PLAN`.)*
150   - [ ] **P1** the native (de-WSL) detector runs on ?1 GPU OS and ?1 CPU/MPS path, producing byte-compatible trajectories (regression vs the WSL runner on a shared clip).
151   - [ ] **P2** a non-technical user can insert a card and get a reel with no terminal, no upload, and no copied original ? verified end-to-end on ?1 OS.
152   - [ ] **P3** signed/notarized installers exist for the greenlit OSes; the bundle passes an MIT/Apache/BSD (+ LGPL-ffmpeg) license audit.
153   - [ ] §7 rows all ? and the headline feasibility verdict is recorded for posterity.
154
155   ## 10. References
156   **Internal code anchors `[verified]`:**
[`tools/generate_highlights.py`](../tools/generate_highlights.py) (local orchestration; `--skip-ai-handoff`, `--wasb-stream`, `--detect-from`, `_WSL_DETECTORS`, `_atomic_publish`) · [`backend/pipeline/detectors/base.py`](../backend/pipeline/detectors/base.py) (`DetectorRunner` ABC + the documented Linux-native de-risk path + `WslCommandMixin`) · [`backend/pipeline/detectors/wasb_runner.py`](../backend/pipeline/detectors/wasb_runner.py) (the WSL adapter to replace) · [`backend/pipeline/detectors/wasb_infer.py`](../backend/pipeline/detectors/wasb_infer.py) (the torch inference core to lift native) · [`backend/pipeline/detectors/tracknet_runner.py`](../backend/pipeline/detectors/tracknet_runner.py) (shared `trajectory_to_action_windows`) · [`backend/pipeline/segmenters/wasb_hybrid.py`](../backend/pipeline/segmenters/wasb_hybrid.py) · `backend/pipeline/stitcher/compiler.py::VideoCompiler` · `pyproject.toml` + `indexer` console entry (#167).
157   **Internal docs:** [`VIDEO_LOCALITY_MODEL.md`](VIDEO_LOCALITY_MODEL.md) (L0 fully-local tier; OQ5 packaging) · [`PLATFORM_ARCHITECTURE.md`](PLATFORM_ARCHITECTURE.md) §3.2 (ComputeBackend seam) · [`OWNED_MODEL_IMPLEMENTATION_PLAN.md`](OWNED_MODEL_IMPLEMENTATION_PLAN.md) (export-clean on-device model) · [`COMMERCIALIZATION.md`](COMMERCIALIZATION.md) (C4 paid-LLM no-train; MIT/Apache/BSD-only) · [`DEFERRED_HARDENING_PLAN.md`](DEFERRED_HARDENING_PLAN.md) (#7 packaging, DONE) · [`TRACKNET_WSL_SETUP.md`](TRACKNET_WSL_SETUP.md) (the WSL setup being superseded for shipped builds).
158   **Measured source:** `current-state.md` 2026-06-18 `COST_SCALING.md` checkpoint (WASB ~3.4× RT on RTX 2070 Super Max-Q, peak VRAM ~5.7?6.0 GB).
159   **External (to validate at P0/P2/P3):** PyTorch MPS backend · onnxruntime (CPU/CoreML/DirectML execution providers) · Tauri vs Electron sizing · ffmpeg LGPL build /

openh264 · Apple Developer-ID notarization · Windows Authenticode/MSIX.
160
161   ---
162
163   ### Changelog
164   - `2026-06-18` ? Created. Scoping doc for the cross-platform desktop app + the de-WSL
native-compute port; all `[design]`. §7 P0 (compute-feasibility spike) named the make-or-break
gate. Owner-gated; nothing built.
165

4. user (2026-06-19T19:17:32.666Z)

1   # UI Alpha Execution Plan ? KhelSutra Guru
2
3   **Status:** ACTIVE · updated 2026-06-10 · **Linked:** `UI_PROPOSAL.md` ·
`HOSTING_PLAN.md` ·
4   `mockups/ui_alpha_mockup.html` · ADR-009 (`archives/decisions/DECISIONS.md`).
5   **Sequencing principle (owner-requested):** build and test **everything locally first**;
6   deployment (DNS/tunnel/Pages/Access) starts only after the local end-to-end gate passes.
7
8   ## Decisions locked
9
10 | Date | Decision |
11 |---|---|
12 | 2026-06-10 | Product name **KhelSutra Guru** · domain **khelsutra.guru** (GoDaddy,
3-yr + protection) |
13 | 2026-06-10 | Separate **private repo `khelsutra`** ("heart and brain"): `web/` SPA
now, control plane at beta; backend enablers stay in the indexer repo |
14 | 2026-06-10 | **Local-first delivery** (this plan's phase structure) |
15 | open | Final "go" on PR-1 coding start (= #16 greenlight) · training opt-in default
OFF · invite list · stack (React+Vite+TS+Tailwind stands unless owner objects) |
16
17   ## Master checklist
18
19   ### G0 ? Setup (now)
20   - [x] Name/domain/repo-split ratified; spelling confirmed via GoDaddy receipt
21   - [x] `khelsutra` repo **scaffolded** at `C:\Users\avidu\Projects\khelsutra` (README,
.gitignore, web/, docs/)
22   - [x] **Owner:** git-init + private GitHub push ? done 2026-06-10:
23   `https://github.com/avidullu/khelsutra` (private; git runs Windows-side per ADR-009)
24   - [x] **Owner:** explicit "go" for PR-1 ? given 2026-06-10
25   - [x] **Owner:** Gemini API key rotated ? 2026-06-10, owner-verified (confirm old-key
deletion in AI Studio)
26
27   ### Phase L ? build & test locally (no cloud; ?6?7 sessions)
28
29   **L1 ? backend enablers** (indexer repo · branch off master · ONE PR each, no stacking ·
offline tests in every PR · GPU/real-video checks = owner):
30   - [x] **PR-1 · B1 async job model (#16):** `jobs` table (user_version 1?2), `POST
/api/process` ? 202 + `job_id`, `GET /api/jobs/{job_id}`, ThreadPoolExecutor worker,
restart/crash state handling ? **MERGED 2026-06-11, PR #87** (CI green: 333 passed; machine-side
real-video checks fold into L3)
31   - [x] **PR-2 · B2+B3 upload & download:** chunked upload (init ? sequential ?95 MB parts
? complete ? MD5; caps + mp4/mov allowlist; per-user dirs deferred to PR-3) + streaming `GET
/api/highlights/{video_id}/{name}` ? **MERGED 2026-06-11, PR #99** (also fixes static-UI
`segment_ids` compile bug + adds browser download link). **Live E2E verified at merge:** 86 MB
real video uploaded in 3 sequential parts and reassembled to MD5 byte-identity; out-of-order 409
/ traversal 400 (handler) + 404 (router) / unknown-id 404 confirmed; compile of a real confirmed
rally ? streamed ? download (3.9 MB reel).
32   - [ ] **PR-3 · B4+B7 identity & exposure:** `owner_email` column + scoping on every
query; Access-JWT/header read with `DEV_USER_EMAIL` env fallback for local dev; CORS narrowed;
rate limit; per-user quota
33   - [ ] **PR-4 · B6 correction loop:** `PATCH /api/segments/{id}` accept/reject/boundary-
nudge ? `candidate_segments.human_label`/`notes` + provenance (who/when); retire
`backend/api/static` (B5)

```
34
35      **L2 ? SPA** (khelsutra repo, `web/`; dev server proxies to `http://localhost:8000`):
36      - [ ] **M1 scaffold:** Vite + React + TS + Tailwind, router, typed API client, branding
(KhelSutra Guru)
37      - [ ] **M2:** A2 Upload (chunked, guidelines checklist) + A3 Job Progress (poll)
38      - [ ] **M3:** A1 Matches + A4 Rally Review (player seek, accept/reject/nudge wired to
PR-4)
39      - [ ] **M4:** A5 Highlights (compile + download) + A6 Report + engine-offline banner
40      - [ ] Unit tests for API client + review-state logic; `npm run build` clean
41
42      **L3 ? local end-to-end (owner's box, real match videos):**
43      - [ ] Full UI flow on localhost: upload ? job ? review/correct ? compile ? download
44      - [ ] `python run_all_tests.py` green; UI polling vs SQLite WAL sanity; WASB/CLI paths
untouched
45      - [ ] **GATE GL:** owner signs off local E2E ? only then deployment work begins
46
47      ### Phase D ? deployment (?2 sessions + ~1 hr owner)
48      - [ ] **D1 (owner):** Cloudflare account ? add khelsutra.guru ? GoDaddy nameserver
switch (HOSTING_PLAN §5) ? scoped API token (§6) ? Pages project connected to GitHub `khelsutra`
(build root `web/`)
49      - [ ] **D2 (Claude):** DNS records; `cloudflared` Windows service on the box ?
`api.khelsutra.guru`; Access apps + email-allowlist policies for `app` & `api`; SPA prod env
50      - [ ] **D3:** deployed smoke E2E (owner-only allowlist); go-live security checklist
(HOSTING_PLAN §7) all green
51      - [ ] **GATE GD:** checklist signed off
52
53      ### Phase R ? rollout
54      - [ ] Wave 1: 5 testers (C13/camera-pilot contacts) + feedback channel (WhatsApp group
or Google Form)
55      - [ ] 2-week retro ? punch list ? widen toward 25 ? beta planning (UI_PROPOSAL §4)
56      - Metrics: matches/week · % rallies corrected · median turnaround · verdict mix · week-2
tester retention
57
58      ## Working agreement
59      Claude (Cowork) designs, codes, writes tests + docs, and leaves a
`scratch/PRn_HANDOFF.md` per
60      work item. **Claude Code executes all git/PR mechanics** (branch, commit, push, PR,
conflict
61      fixes, merges after owner review) by reading that handoff ? the owner never copy-pastes
62      commands (ratified 2026-06-11 after PR-1/#88 friction; supersedes the command-block
pattern).
63      Cowork's sandbox cannot run git on mounts (ADR-009) and CI runs the offline suite on
every PR.
64      One PR per item; no stacking; never push to master directly. PowerShell note: no `&&`.
65
66      ## Risks & mitigations
67      Box uptime (offline banner, alpha-acceptable) · Gemini 429 burst-throttle (serial
workers,
68      `ai_max_workers=1`, 6-attempt backoff ? learned 2026-06-10) · CF free-tier 100 MB body
cap
69      (chunked upload design) · single-GPU box (alpha default = Gemini path; WASB jobs queue-
depth 1) ·
70      .guru renewal cost ?18.8k/3-yr (optional: transfer to Cloudflare Registrar at-cost after
the
71      60-day registration lock; domain-protection may need toggling for transfer).
72
73      ## Session map (default order)
74      S1?S2 PR-1 · S3 PR-2 · S4 PR-3 · S5 PR-4 · S6 M1+M2 · S7 M3 · S8 M4 · S9 L3 E2E + fixes
.
75      S10 D1?D3 · S11 wave-1 launch
76
```

5. assistant (2026-06-19T19:17:36.553Z)

Both docs read in full. The idea is not anticipated by either doc, but there are clean seams

where it would slot. Let me verify the one cross-reference both docs lean on ? the `VIDEO_LOCALITY_MODEL.md` tier model ? since both anchor to it and the desktop doc explicitly realizes "the L0 tier."

6. user (2026-06-19T19:17:38.696Z)

7:`PLATFORM_ARCHITECTURE.md` §3 (future StorageBackend/ComputeBackend registries),
21:under the cap the file moves as sequential ?90 MB parts through a free-tier edge
25:miserable everywhere; expecting academies to manage S3/GCS buckets is a
46:must cross the network for this tier of service."
51:ratios for a 4K source ? measure on a real file before committing to any tier.
55:| **L3 ? full upload** (today's flow) | everything | 10 GB (blocked: 4 GB cap) | paid edge tier + resumable uploads + server disk + retention policy | built, capped, wrong for big files |
56:| **L2 ? proxy upload** | a timeline-identical ?1080p re-encode | ~1?2.5 GB | a tiny client prep step (ffmpeg) ? **the exact frame-identical contract we already built and verified as collector `prep-annotation-proxy`** (same fps, same frame count, parity-checked, MD5?MD5 manifest) | contract proven; client wrapper + server acceptance = new work |
57:| **L1 ? chunks/candidates upload** | only the ?1280px AI chunks (or just candidate windows) the Gemini path needs anyway | ~0.3?1 GB | client-side chunk+downscale (ffmpeg, CPU); server becomes a **GPU-less orchestrator** | the server half exists (it's the gemini segmenter's own flow, relocated) |
58:| **L1-direct** | chunks go **straight from the client to Gemini** ? zero video bytes through our server | ~0 through us | per-user key custody / metering / cost ceilings (OQ4) | idea only |
59:| **L0 ? fully local** | nothing (calls to Google only if the Gemini path is used) | 0 | the owned on-device model (roadmap north star) or a local GPU + our stack | not alpha-feasible; the long-term answer |
60:| **L4 ? sneakernet** | a hard drive | 0 over network | a person (the field associate / owner) | how the pilot actually works today |
62:**The honest "no upload" answer:** *strictly* no bytes leave only at L0 with the
63:owned model (and even L0 leaks ?1280px chunks to Google while the Gemini path is
65:answer is **L2 or L1: don't eliminate the upload, shrink it 5?30×** ? at which
71:- **L1/L2 make the serving box GPU-less for the Gemini tier** ? ffmpeg + API
77:- **The correction loop survives** at every tier ?L1: labels are intervals on a
80:- **video_id keying:** the indexer keys by MD5 of the file it processed. At L2
91: (2026-06-10 quality table) and at L1/L2 needs **no GPU at all** ? a CPU box,
96: model has a ship-gate-passing reason to be served. L2/L1 turn this from a
99: disk); settle retention (OQ1) before volume arrives; the free edge tier stays
100: adequate precisely because L2 keeps payloads under the existing caps.
115: back on a drive (L4), we proxy + process + return a reel ? the same motion
116: L2 will automate. Every friction the associate hits there (file handoff,
127:| OQ1 | **Retention policy per artifact class** ? original (only L3 has it), proxy, chunks, frames/trajectories, Gemini remote files, intervals/labels | intervals/labels are the asset (keep forever, tiny); originals/proxies need an explicit keep-while-active + delete-on-request rule; Gemini remote-file GC already in BACKLOG |
128:| OQ2 | **Which file's MD5 is the video's identity** at L2/L1 | lean: the processed file's MD5 (proxy), with original_md5 carried in metadata ? the collector manifest pattern, promoted to a product rule |
129:| OQ3 | **Consent language for the Gemini path** ? even "local" modes ship 1280px chunks to Google | must be plain in the academy-facing terms; paid-tier no-training posture to be cited |
130:| OQ4 | **Key custody for L1-direct** ? client calls Gemini with whose key? per-user metering, abuse, cost ceiling per video | unsolved; this is what separates L1 from L1-direct |
132:| OQ6 | **What does the review UI play** at each tier | L2 streams the proxy (good); L1 has only chunks ? review degrades ? this quietly argues L2 over L1 as the default |
134:| OQ8 | **Real academy uplink numbers** ? the L2-vs-L3 case rests on bandwidth assumptions | C13 visits can note connectivity informally; don't redesign around guesses |
135:| OQ9 | **Recurring volume** ? an academy recording daily generates 10 GB/day; even L2 accumulates | retention (OQ1) + maybe future delta thinking; not an alpha problem |
136:| OQ10 | **If L3 must survive** (users who refuse any client-side step) | needs resumable uploads + a paid edge tier + a raised cap; treat as the exception path, not the design center |
140:**Default the alpha to L2 (proxy upload):** a one-command client prep step built
145:**L1/L1-direct is the fast-follow** once OQ4 has an answer; **L0 is the owned-
147:a person with a hard drive is a fine storage backend for n=3 academies.

7. assistant (2026-06-19T19:17:44.106Z)

This is highly relevant. Line 25 explicitly mentions S3/GCS buckets (dismissively) and the whole tier model (L0?L4) is the spine. Let me confirm the `PLATFORM_ARCHITECTURE.md` StorageBackend registry that both docs reference, since that's where a connector would most naturally live.

8. user (2026-06-19T19:17:46.050Z)

15:browsable by history**, and choose **where their data lives** (Google Drive / cloud
16:bucket / local) and **where the compute runs** (our GPU / their VM). The organizing
17:principle is to **decouple the layers** ? identity, ingestion, storage, orchestration,
19:**pluggable interfaces**, reusing the registry pattern the codebase already uses for
41:> Spiideo offer true BYO-storage or BYO-compute ? they're vendor-cloud SaaS
48:> first infra slice (an async job model) is the on-hold #16 in
58:| Storage | local `filepath` string; output to `output_dir` | no abstraction; can't BYO Drive/bucket; raw video must be on the box |
60:| Execution | **async job model shipped** ? `/api/process` returns **202 + job_id**, a single-worker `ThreadPoolExecutor` drains the `jobs` table, `GET /api/jobs/{id}` polls status | single-tenant + local-only; can't dispatch to a user's VM/GPU yet |
61:| Pipeline | **already pluggable** (segmenter/provider/validator/sport/annotation registries) | reuse as-is ? this layer is the model for the rest |
62:| API | `/api/process`, `/api/query`, `/api/compile`, `/api/config`; no auth | needs auth, per-tenant scoping, async job + history endpoints |
66:`backend/main.py` (endpoints + the async job worker ? `/api/process` ? 202 + job_id,
67:`GET /api/jobs/{id}`), the registry pattern in
68:`backend/pipeline/segmenters/base.py::SegmenterRegistry` and `backend/providers/base.py`.
74:elsewhere (even in the user's own cloud).
92:                ? Storage (local upload | Google Drive | S3 | ?
93:                ? GCS | Azure) ? raw video + highlight outputs ?
101:is delivered through the existing **AI-provider registry with a paid fallback**.
103:## 3. Pluggable abstractions (extend the existing registry pattern)
105:The codebase already proves the model: an `ABC` + a `Registry` singleton with
106:`register/get/list_available` (e.g. `SegmenterRegistry`). Add two more registries with
109:### 3.1 `StorageBackend` registry
114:class StorageBackend(ABC):
125: local / S3 (`s3fs`) / GCS (`gcsfs`) / Azure (`adlfs`) + Google Drive via `gdrivefs`,
126: Dropbox via `dropboxdrivefs`), reaching for **`smart_open`** as the robust large-MP4
127: byte-streaming primitive. Storage becomes one more registry. (Treat Drive/Dropbox as
128: add-ons ? less battle-tested than `s3fs`/`gcsfs`; cover with integration tests.)
129:- **Backends:** `local` (upload dir), `gdrive`, `s3`, `gcs`, `azure`. **BYO-storage**:
130: the tenant registers their own bucket/Drive; raw video **never leaves their account**.
131:- **Auth ? never persist users' long-lived keys.** For buckets, have the user create an
132: IAM role you **`AssumeRole`** into, scoped to one bucket/prefix, and mint **presigned
133: URLs** so raw video moves **browser ? the user's bucket directly, never through our
134: servers**. For Drive, OAuth with the narrow **`drive.file`** scope; for academies, a
143:### 3.2 `ComputeBackend` / `ExecutionBackend` registry
152: user's cloud credentials** and needs **no inbound access** to their network. This
156:- For users who want the platform to **provision GPUs in their *own* cloud account on
169: **storage** stays in the user's bucket (signed URLs) while compute runs on **our** GPUs
177:The existing `segmenter` / `provider` / `validator` / `sport` / `annotation` registries
178:are the proven precedent and stay as the **pipeline** layer. The `annotation` registry +
223:## 5. Orchestration / async jobs
226:[`DEFERRED_HARDENING_PLAN.md`](DEFERRED_HARDENING_PLAN.md) #16** (async job model) ? build
235: for its tenant, resolves input via the tenant's `StorageBackend`, runs the pipeline,
243: - **Start simplest:** **Arq** (async-native, natural FastAPI fit) or even a **plain
245:    async slice, with the lowest operational complexity and the easiest debugging.
279:> serving / fallback* only ? never as a *training* data source.** Harvesting their outputs
286:The codebase already abstracts the cloud AI behind `provider_registry` /
288:- Every generative capability routes through the **provider registry**.
302: rally/segment/event base VLM via   quality + the   ComputeBackend;
307:**Data** supplies labels (the `annotation` registry + `candidate_segments` +
356:| **ms-swift** (ModelScope) | Self-managed (rented/own GPU) | **Native video+audio**; Qwen3-VL/Omni, 300+ MLLMs | any incl **7B** | ? DPO/GRPO | ? full ownership | **? video-QA; slots into BYO-compute (§3.2)** |
365:*(Enterprise clouds ? Databricks Mosaic / SageMaker / Vertex on open models ? give full
370: Unsloth) on a **rented GPU** (RunPod / Lambda / Modal / SkyPilot-to-your-cloud): native
372: **ComputeBackend / BYO-compute** seam (§3.2). **Likely a better fit than Tinker** for

383: backend is swappable ? same ethos as the provider/registry pattern. Don't get locked in.
386:ingestion + max frames per example; (2) **weight export** + a **commercial DPA** covering
401:**Engine (don't reinvent):** **`ms-swift`** already delivers this for VLMs ? CLI-driven,
408:1. **Build dataset** from golden labels ? clip?QA pairs (reuse the `annotation` registry +
410:2. **Fine-tune** via ms-swift on a **ComputeBackend** GPU (§3.2), run as an orchestration
443: **explicitly-contributed, licensed** corpus ? *not* on whatever sits in a tenant's bucket
475:| **P0** | ? **DONE ? Async job model, single-tenant** (DEFERRED #16) | `jobs` table via `user_version` ladder; single-worker `ThreadPoolExecutor`; `/api/process` ? 202 + job_id; `GET /api/jobs/{id}` status |
477:| **P2** | **`StorageBackend` registry + BYO-storage** | new ABC+registry; `local`?`gdrive`/`s3`/`gcs`; replace `video_path` with `StorageRef`; encrypted per-tenant creds |

9. assistant (2026-06-19T19:17:51.868Z)

I now have the full picture. The `PLATFORM_ARCHITECTURE.md` doc is where BYO-storage genuinely lives (a `StorageBackend` registry, §3.1, with gdrive/s3/gcs/azure/dropbox backends and BYO-storage as an explicit goal). But my assigned scope is the two surface docs ? `DESKTOP_APP_PLAN.md` and `UI_ALPHA_EXECUTION_PLAN.md` ? and neither of those anticipates a cloud-storage *source* or a watch/autosync daemon. Let me check the "autosync" / "watch" dimension specifically, which is the part even PLATFORM_ARCHITECTURE doesn't cover.

10. user (2026-06-19T19:17:54.208Z)

docs/PLATFORM_ARCHITECTURE.md:60:| Execution | **async job model shipped** ? `/api/process` returns **202 + job_id**, a single-worker `ThreadPoolExecutor` drains the `jobs` table, `GET /api/jobs/{id}` polls status | single-tenant + local-only; can't dispatch to a user's VM/GPU yet |
docs/PLATFORM_ARCHITECTURE.md:141: backend wraps today's behavior so nothing breaks; adapters land incrementally.
docs/PLATFORM_ARCHITECTURE.md:154: outbound-only HTTPS long-poll). This pull-based runner is the **cheapest, most
docs/PLATFORM_ARCHITECTURE.md:163: their own VM/GPU* is a meaningful ongoing load (updates, monitoring, firewall/NAT, the
docs/PLATFORM_ARCHITECTURE.md:231: timestamps`. `/api/process` returns **202 + job_id**; `/api/jobs/{id}` polls.
docs/VIDEO_LOCALITY_MODEL.md:99: disk); settle retention (OQ1) before volume arrives; the free edge tier stays
docs/VIDEO_LOCALITY_MODEL.md:135:| OQ9 | **Recurring volume** ? an academy recording daily generates 10 GB/day; even L2 accumulates | retention (OQ1) + maybe future delta thinking; not an alpha problem |

11. assistant (2026-06-19T19:17:59.168Z)

Confirmed. No "watch/autosync daemon" concept exists anywhere ? even the platform doc treats storage as a pull-on-demand `StorageBackend.open()`, not a watched, event-driven source. The "footage lands ? reels appear automatically" loop is genuinely new. I have everything I need to return an evidence-based verdict scoped to the two assigned docs.

12. user (2026-06-19T19:18:50.784Z)

Structured output provided successfully

13. assistant (2026-06-19T19:19:03.934Z)

I've completed the assessment and returned the structured object.

Summary of findings: Neither assigned doc anticipates the BYO-storage + autosync idea.

- **`docs/DESKTOP_APP_PLAN.md`** assumes the source is removable physical media only ? "User plugs the GoPro/phone SD card or camera USB into their laptop" (§1) ? or "a chosen [local] folder" (D2). Its only "watcher" is a per-OS hardware device-insert watcher (§4.1: Win WM_DEVICECHANGE, macOS DiskArbitration, Linux udev), not a cloud-folder change watcher. Its D1/D2 invariants ("no upload, never copy the originals, fully on-device") directly conflict with

pulling GBs from a cloud account.

- **`docs/UI_ALPHA_EXECUTION_PLAN.md`** has exactly one ingest path ? manual chunked browser upload (PR-2 B2/B3, mp4/mov-allowlisted). No "connect external storage" screen, no connector backend enabler, none budgeted in the S1?S11 session map.

Where it actually belongs (out of assigned scope but worth flagging):
`docs/PLATFORM_ARCHITECTURE.md` §3.1 already specs the *source* half ? a `StorageBackend` registry with `local/gdrive/s3/gcs/azure` backends and explicit "BYO-storage" plus the right auth posture (narrow `drive.file` OAuth, IAM AssumeRole, presigned URLs) ? scheduled as roadmap **P2, not yet built**. But the **autosync/watch daemon** half ("footage lands ? reels appear automatically") exists in **no doc**: the shipped async job model is request-driven (`POST /api/process`), with no poller, webhook, change-feed, or arrival-dedup ledger. That loop is a genuinely new component.

One persona caveat the docs surface: `VIDEO_LOCALITY_MODEL.md` line 25 calls "expecting academies to manage S3/GCS buckets" a non-starter for the non-technical target user ? so a consumer-cloud "watch my Drive/OneDrive folder" framing fits better than raw BYO-bucket.

Relevant file paths (all absolute):
- `C:\Users\avidu\Projects\badminton-highlight-indexer\docs\DESKTOP_APP_PLAN.md`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\docs\UI_ALPHA_EXECUTION_PLAN.md`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\docs\PLATFORM_ARCHITECTURE.md` (§3.1 StorageBackend ? where the connector belongs)
- `C:\Users\avidu\Projects\badminton-highlight-indexer\docs\VIDEO_LOCALITY_MODEL.md` (L0?L4 tier model + line 25 persona caveat)